

Documentación Proyecto Individual

Ejercicio a. Clasificación de texturas en imágenes

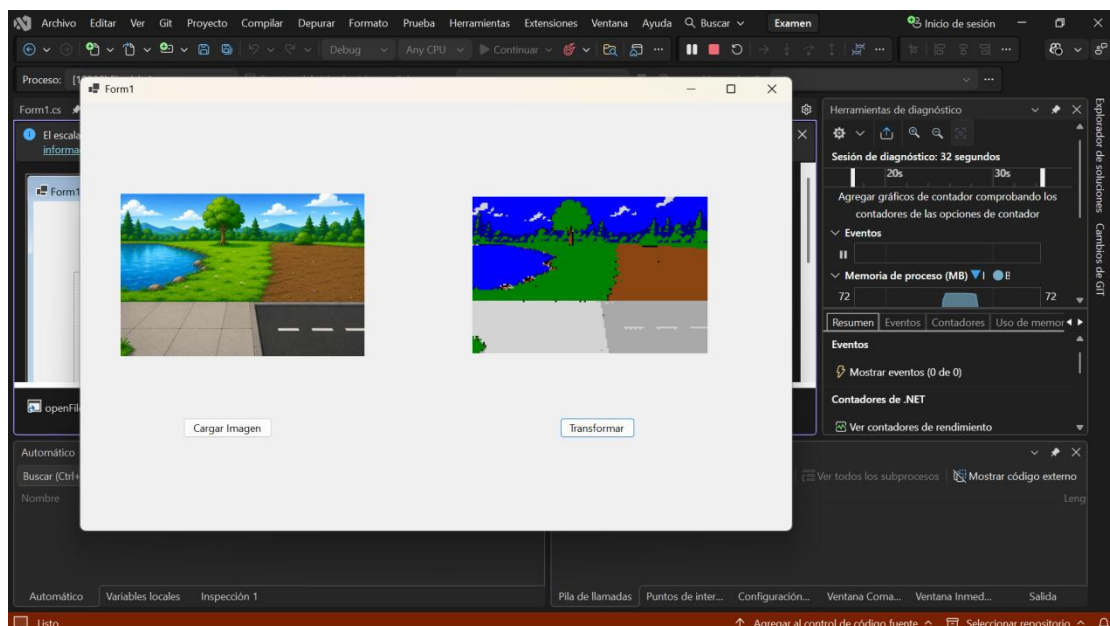
Se desarrolló una aplicación en Windows Forms utilizando C# con el objetivo de clasificar diferentes tipos de superficies presentes en una imagen, como césped, tierra, cemento y asfalto.

La implementación se realizó mediante la carga de imágenes desde el sistema utilizando un OpenFileDialog, permitiendo al usuario seleccionar una imagen desde su equipo. Una vez cargada, la imagen se muestra en un PictureBox para su visualización.

Para el procesamiento de la imagen se utilizó la clase Bitmap, lo que permitió acceder a los píxeles de forma individual. A partir de este acceso, se analizaron los valores de color RGB de cada pixel con el fin de obtener características representativas de la textura de la imagen.

La clasificación se realizó mediante la evaluación de los valores promedio de color y la variación de intensidades. Dependiendo de estos valores, la imagen o sus regiones fueron asociadas a una categoría específica de superficie. Por ejemplo, los tonos verdes predominantes se asociaron a césped, los tonos marrones a tierra, y los tonos grises a cemento o asfalto, diferenciándolos según la intensidad.

El resultado del procesamiento se mostró en un segundo PictureBox, permitiendo visualizar la imagen analizada o la clasificación obtenida.



```
using System;using System.Drawing;using System.Windows.Forms;
namespace Examen
{
    public partial class Form1 : Form
    {
        Bitmap imagenOriginal;
```

```

public Form1()
{
    InitializeComponent();
}

// Botón para cargar imagen
private void button1_Click(object sender, EventArgs e)
{
    OpenFileDialog dialogo = new OpenFileDialog();
    dialogo.Filter = "Imágenes|*.jpg;*.jpeg;*.png;*.bmp";

    if (dialogo.ShowDialog() == DialogResult.OK)
    {
        imagenOriginal = new Bitmap(dialogo.FileName);
        pictureBox1.Image = imagenOriginal;
        pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;
    }
}

// Botón para clasificar texturas
private void button2_Click(object sender, EventArgs e)
{
    if (imagenOriginal == null)
    {
        MessageBox.Show("Primero cargue una imagen.");
        return;
    }

    Bitmap resultado = ClasificarTexturas(imagenOriginal);
    pictureBox2.Image = resultado;
    pictureBox2.SizeMode = PictureBoxSizeMode.Zoom;
}

private Bitmap ClasificarTexturas(Bitmap imagen)
{
    Bitmap salida = new Bitmap(imagen.Width, imagen.Height);

    int bloque = 10; // Tamaño del bloque de análisis

    for (int y = 0; y < imagen.Height; y += bloque)
    {
        for (int x = 0; x < imagen.Width; x += bloque)
        {
            int sumaR = 0;
            int sumaG = 0;
            int sumaB = 0;
            int contador = 0;

            for (int j = y; j < y + bloque && j < imagen.Height; j++)
            {
                for (int i = x; i < x + bloque && i < imagen.Width;
i++)
                {
                    Color pixel = imagen.GetPixel(i, j);

                    sumaR += pixel.R;
                    sumaG += pixel.G;
                    sumaB += pixel.B;
                    contador++;
                }
            }

            int promedioR = sumaR / contador;
            int promedioG = sumaG / contador;
            int promedioB = sumaB / contador;

```

```

        Color colorClase = ClasificarColor(promedioR, promedioG,
promedioB);

        for (int j = y; j < y + bloque && j < imagen.Height; j++)
        {
            for (int i = x; i < x + bloque && i < imagen.Width;
i++)
            {
                {
                    salida.SetPixel(i, j, colorClase);
                }
            }
        }

        return salida;
    }

    private Color ClasificarColor(int r, int g, int b)
    {
        int brillo = (r + g + b) / 3;

        // Agua: tonos azules o azul verdosos
        if (b > r + 20 && b > g + 10)
        {
            return Color.Blue;
        }

        // Agua verdosa u oscura
        if (g > r + 10 && b > r + 10 && brillo < 130)
        {
            return Color.DarkBlue;
        }

        // Césped: predominio del verde
        if (g > r + 20 && g > b + 20)
        {
            return Color.Green;
        }

        // Tierra: tonos marrones
        if (r > 80 && g > 50 && b < 80 && r > b && g > b)
        {
            return Color.SaddleBrown;
        }

        // Cemento: gris claro
        if (Math.Abs(r - g) < 25 && Math.Abs(g - b) < 25 && brillo >= 120)
        {
            return Color.LightGray;
        }

        // Asfalto: gris oscuro
        if (Math.Abs(r - g) < 30 && Math.Abs(g - b) < 30 && brillo < 120)
        {
            return Color.DarkGray;
        }

        // No clasificado
        return Color.Black;
    }

    private void pictureBox1_Click(object sender, EventArgs e)
    {
    }
}

```

```

        private void pictureBox2_Click(object sender, EventArgs e)
        {

        }

        private void openFileDialog1_FileOk(object sender,
System.ComponentModel.CancelEventArgs e)
        {

        }

        private void Form1_Load(object sender, EventArgs e)
        {

        }
    }
}

```

Instalación y ejecución del programa

1. Instalación de Visual Studio

- Descarga e instala Visual Studio 2019 o superior desde la página oficial de Microsoft.
- Durante la instalación, asegúrate de seleccionar el paquete:
 - “Desarrollo de escritorio con .NET (Windows Forms)”

2. Abrir el proyecto

- Descomprime la carpeta del proyecto en tu computadora.
- Abre Visual Studio.
- Haz clic en “Abrir un proyecto o una solución”.
- Busca y abre el archivo con extensión .sln (solución del proyecto).
- Espera a que Visual Studio cargue todos los archivos.

3. Configurar y ejecutar

- En el Explorador de soluciones, asegúrate de que el proyecto llamado Examen esté seleccionado como proyecto principal.
- Haz clic en el botón Start o presiona F5 para ejecutar el programa.
- Si todo está correcto, se abrirá una ventana de Windows Forms.

4. Uso del programa

El sistema funciona con dos botones principales:

Botón “Cargar imagen”

- Abre un explorador de archivos.
- Permite seleccionar imágenes en formato: .jpg, .jpeg, .png o .bmp.
- La imagen seleccionada se mostrará en el primer recuadro (pictureBox1).

Botón “Clasificar texturas”

- Procesa la imagen cargada.
- Analiza la imagen en bloques de 10x10 píxeles.
- Clasifica cada zona según el color promedio.

5. Resultado del procesamiento

La imagen procesada se muestra en el segundo recuadro (pictureBox2) con la siguiente clasificación de colores:

- Agua: tonos azules o azul verdosos
- Agua verdosa u oscura
- Césped o vegetación
- Tierra
- Cemento
- Asfalto
- Zonas no clasificadas

Ejercicio b. Implementación de un Filtro de Suavizado (Promedio 3x3)

Se desarrolló una aplicación en Windows Forms utilizando C# con el objetivo de aplicar un filtro de suavizado a una imagen, reduciendo el ruido y suavizando las transiciones de color.

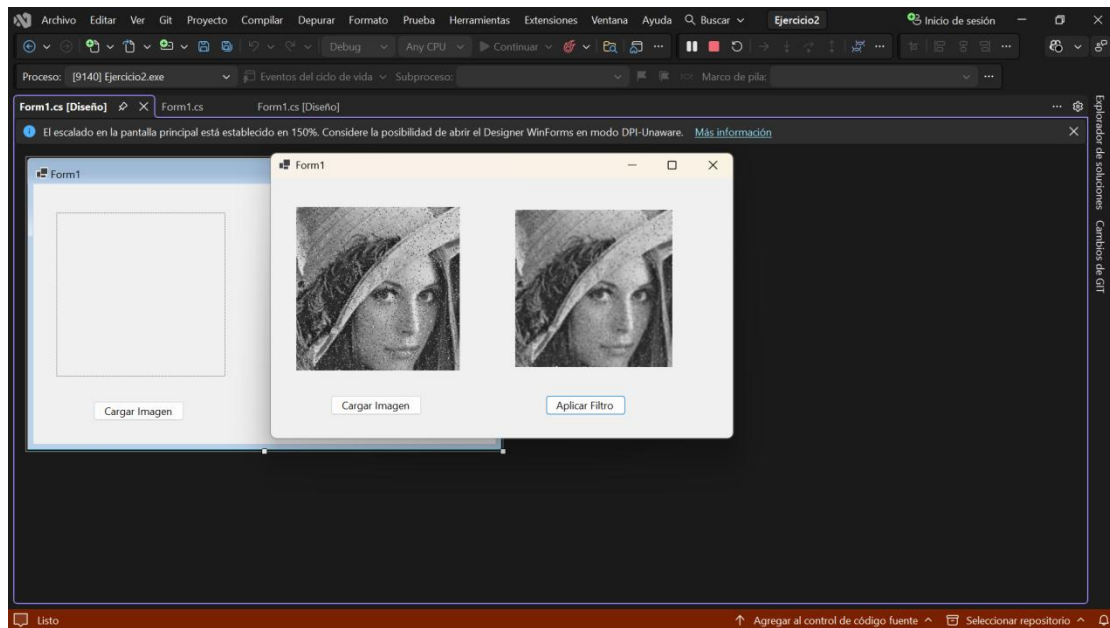
La implementación permite al usuario cargar una imagen desde su equipo mediante un OpenFileDialog, la cual se almacena en un objeto Bitmap y se muestra en un PictureBox para su visualización.

El filtro de suavizado se implementó mediante un algoritmo de promedio de vecinos utilizando una ventana de 3x3 píxeles. Para cada píxel de la imagen (exceptuando los bordes), se recorren sus 8 píxeles vecinos y el propio píxel central, obteniendo los valores de color RGB. Luego, se calcula el promedio de cada canal de color (R, G y B) dividiendo la suma total entre 9.

El nuevo valor promedio obtenido se asigna a un nuevo Bitmap, generando así una versión suavizada de la imagen original. Este proceso permite disminuir variaciones bruscas de color y reducir el ruido visual.

Para evitar distorsiones en los bordes de la imagen, estos se mantienen sin modificación, copiando directamente los valores originales de la imagen fuente.

El resultado final se muestra en un segundo PictureBox, permitiendo comparar la imagen original con la imagen suavizada.



```
using System;using System.Drawing;using System.Windows.Forms;
namespace Ejercicio2
{
    public partial class Form1 : Form
    {
        Bitmap imagenOriginal;

        public Form1()
        {
            InitializeComponent();
        }

        private void pictureBox1_Click(object sender, EventArgs e)
        {
        }

        private void pictureBox2_Click(object sender, EventArgs e)
        {
        }

        // Botón para cargar imagen
        private void button1_Click(object sender, EventArgs e)
        {
            OpenFileDialog abrir = new OpenFileDialog();
            abrir.Filter = "Archivos de imagen|*.jpg;*.jpeg;*.png;*.bmp";

            if (abrir.ShowDialog() == DialogResult.OK)
            {
                imagenOriginal = new Bitmap(abrir.FileName);
                pictureBox1.Image = imagenOriginal;
                pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;
            }
        }

        // Botón para aplicar filtro promedio 3x3
        private void button2_Click(object sender, EventArgs e)
        {
            if (imagenOriginal == null)
            {
                MessageBox.Show("Primero debes cargar una imagen.");
                return;
            }
        }
    }
}
```

```

    }

    Bitmap imagenSuavizada =
    AplicarFiltroPromedio3x3(imagenOriginal); ;

    for (int i = 0; i < 4; i++)
    {
        imagenSuavizada = AplicarFiltroPromedio3x3(imagenOriginal);
    }

    pictureBox2.Image = imagenSuavizada;
    pictureBox2.SizeMode = PictureBoxSizeMode.Zoom;
}

private Bitmap AplicarFiltroPromedio3x3(Bitmap imagen)
{
    Bitmap resultado = new Bitmap(imagen.Width, imagen.Height);

    for (int y = 1; y < imagen.Height - 1; y++)
    {
        for (int x = 1; x < imagen.Width - 1; x++)
        {
            int sumaR = 0;
            int sumaG = 0;
            int sumaB = 0;

            // Ventana de 3x3 píxeles
            for (int j = -1; j <= 1; j++)
            {
                for (int i = -1; i <= 1; i++)
                {
                    Color pixel = imagen.GetPixel(x + i, y + j);

                    sumaR += pixel.R;
                    sumaG += pixel.G;
                    sumaB += pixel.B;
                }
            }

            int promedioR = sumaR / 9;
            int promedioG = sumaG / 9;
            int promedioB = sumaB / 9;

            resultado.SetPixel(x, y, Color.FromArgb(promedioR,
promedioG, promedioB));
        }
    }

    // Copiar bordes originales para que no queden negros
    for (int x = 0; x < imagen.Width; x++)
    {
        resultado.SetPixel(x, 0, imagen.GetPixel(x, 0));
        resultado.SetPixel(x, imagen.Height - 1, imagen.GetPixel(x,
imagen.Height - 1));
    }

    for (int y = 0; y < imagen.Height; y++)
    {
        resultado.SetPixel(0, y, imagen.GetPixel(0, y));
        resultado.SetPixel(imagen.Width - 1, y,
imagen.GetPixel(imagen.Width - 1, y));
    }

    return resultado;
}
}

```

}

Instalación y ejecución del programa

1. Instalación de Visual Studio

- Descargar e instalar **Visual Studio 2019 o superior** desde la página oficial de Microsoft.
- Durante la instalación, seleccionar la carga de trabajo:
 - **“Desarrollo de escritorio con .NET (Windows Forms)”**

2. Abrir el proyecto

- Descomprimir la carpeta del proyecto en el equipo local.
- Abrir **Visual Studio**.
- Seleccionar la opción **“Abrir un proyecto o una solución”**.
- Buscar y abrir el archivo con extensión **.sln** correspondiente al proyecto.
- Esperar a que el entorno cargue completamente todas las dependencias.

3. Compilar y ejecutar

- Verificar que el proyecto esté configurado como proyecto de inicio.
- Presionar el botón **Start (▶)** o la tecla **F5** para compilar y ejecutar la aplicación.
- Se abrirá una ventana de Windows Forms con la interfaz del programa.

4. Uso del programa

El sistema permite aplicar un filtro de suavizado (promedio 3x3) a una imagen cargada.

Botón “Cargar imagen”

- Abre un explorador de archivos.
- Permite seleccionar imágenes en formato **.jpg, .jpeg, .png o .bmp**.
- La imagen seleccionada se muestra en el primer PictureBox.

Botón “Aplicar filtro promedio 3x3”

- Aplica un filtro de suavizado a la imagen cargada.
- El algoritmo recorre la imagen en ventanas de 3x3 píxeles.
- Para cada píxel se calcula el promedio de los valores RGB de sus 8 vecinos y el píxel central.
- Esto genera una imagen más suave, reduciendo el ruido y las variaciones bruscas de color.
- La imagen resultante se muestra en el segundo PictureBox.

5. Funcionamiento del algoritmo

El filtro realiza los siguientes pasos:

- Recorre la imagen píxel por píxel (excepto bordes).
- Suma los valores de los canales rojo, verde y azul en una ventana de 3x3.
- Calcula el promedio dividiendo entre 9.
- Asigna el nuevo color al píxel correspondiente en una nueva imagen.
- Copia los bordes originales para evitar distorsiones.

6. Flujo de uso recomendado

1. Ejecutar el programa con F5
2. Presionar “Cargar imagen”
3. Seleccionar una imagen del equipo
4. Presionar “Aplicar filtro promedio 3x3”
5. Visualizar la imagen suavizada en el segundo recuadro

Ejercicio C. Cover de la Vaca Lola

Descripción general del proyecto

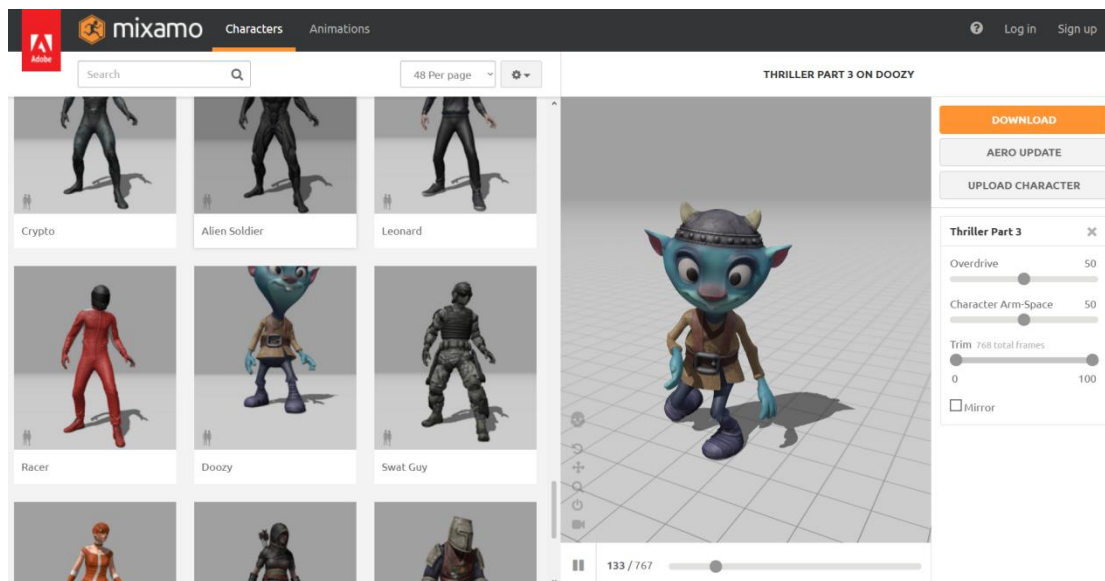
El proyecto consistió en la creación de una escena animada en Unity utilizando un personaje 3D con una animación de baile, acompañado por audio generado y editado mediante herramientas de inteligencia artificial. El objetivo fue integrar un modelo animado dentro de Unity y complementarlo con una pista de voz y música infantil instrumental, generando un resultado multimedia completo.

Para lograrlo, se utilizó Mixamo para obtener el modelo 3D y su animación, Unity para la implementación visual y animada del personaje, y Google Colab junto con modelos de inteligencia artificial para la generación y edición del audio.

Obtención del modelo 3D y animación

Primero se ingresó a la plataforma Mixamo, donde se seleccionó un personaje 3D y se le asignó una animación de baile. Mixamo permite descargar modelos tridimensionales ya preparados con rigging y animaciones, lo cual facilita su uso dentro de motores de desarrollo como Unity.

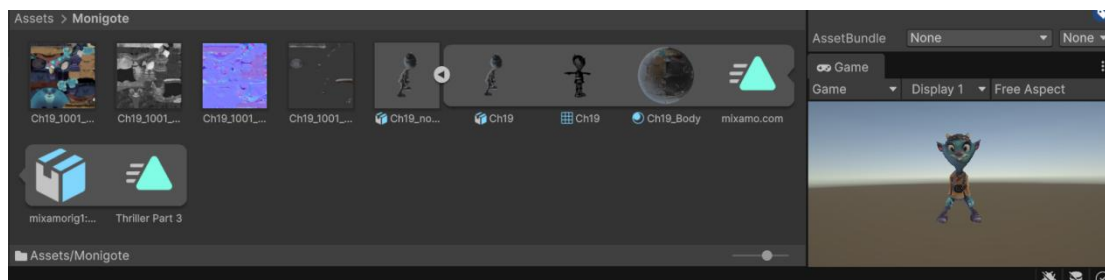
Después de seleccionar el personaje y la animación deseada, se descargó el archivo correspondiente para poder importarlo posteriormente dentro del proyecto de Unity. La animación descargada correspondía a un baile, el cual fue utilizado como movimiento principal del personaje.

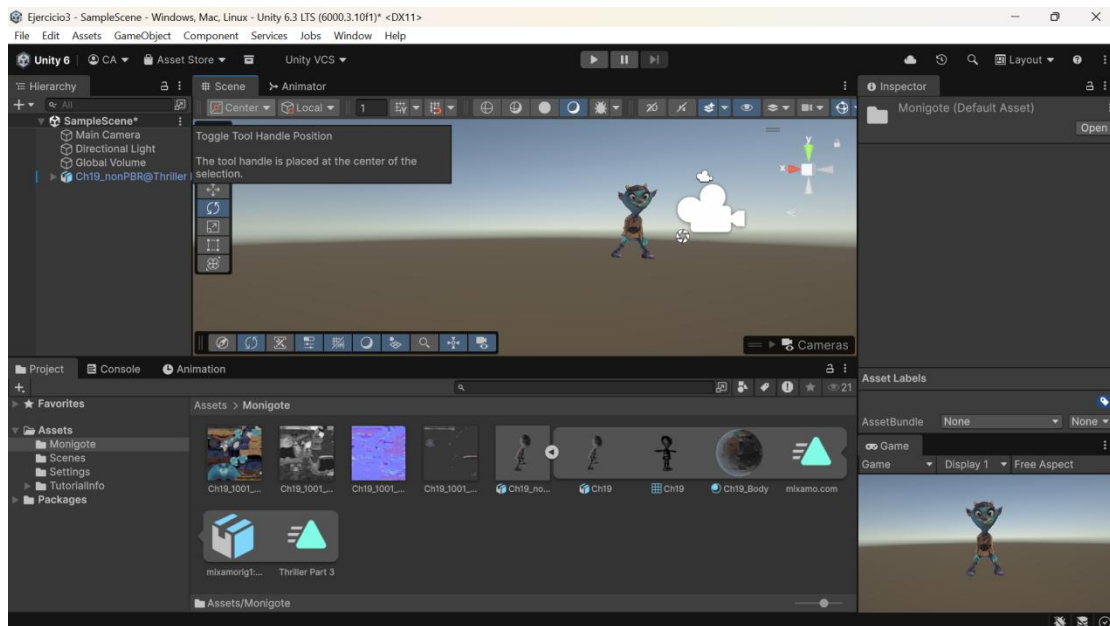


Importación del modelo en Unity

Una vez descargado el modelo desde Mixamo, se importó dentro del proyecto de Unity. Para ello, el archivo del personaje se agregó a la carpeta de Assets del proyecto. Unity reconoció el modelo y sus componentes, permitiendo configurarlo para su uso dentro de la escena.

Luego, el personaje fue colocado en el escenario principal del proyecto. Se revisó que el modelo se visualizara correctamente y que la animación estuviera disponible para ser utilizada dentro del sistema de animación de Unity.



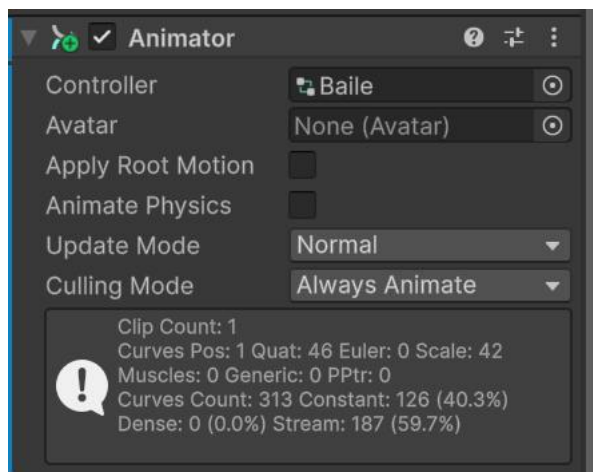
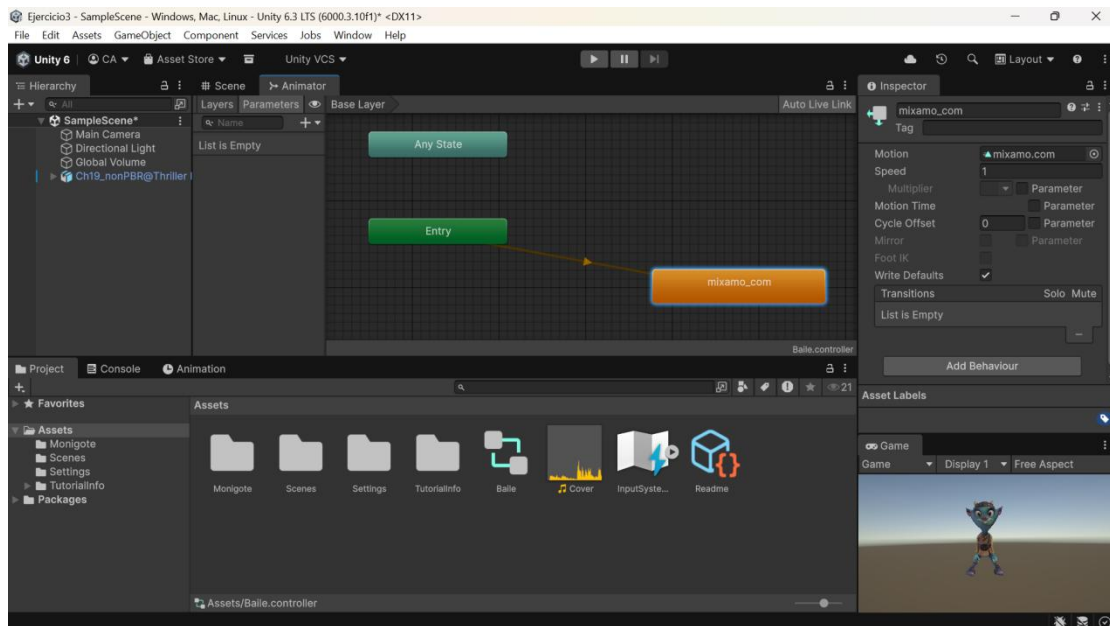


Configuración del Animator

Después de importar el modelo, se trabajó con el componente Animator de Unity. En este componente se añadió la animación de baile descargada desde Mixamo.

El objetivo de esta configuración fue permitir que el personaje ejecutara el baile de manera automática. Para esto, se asignó la animación al Animator Controller y se configuró como animación principal del personaje.

También se configuró la animación para que se reprodujera en bucle, de manera que el personaje continuara bailando durante toda la ejecución del programa sin detenerse. Esto permitió que la escena tuviera movimiento constante y que el personaje permaneciera activo mientras se reproducía el audio.



Preparación del entorno en Google Colab

Para la parte de audio se utilizó Google Colab, ya que permite ejecutar código Python y utilizar modelos de inteligencia artificial sin necesidad de instalarlos directamente en una computadora local.

Primero se instalaron las dependencias necesarias para trabajar con clonación de voz y procesamiento de audio. Entre las librerías utilizadas se encuentran qwen-tts, torch, soundfile, pydub y herramientas de reproducción y descarga de archivos dentro del notebook.

También se verificó si el entorno tenía acceso a GPU mediante CUDA. Esta verificación fue importante porque los modelos de inteligencia artificial pueden tardar mucho más cuando se ejecutan únicamente con CPU. En caso de existir GPU disponible, el código la utiliza automáticamente para acelerar el procesamiento.

```
pip install accelerate==1.12.0
```

```

# Instalar dependencias
!pip install -U qwen-tts
!pip install -U flash-attn --no-build-isolation
!pip install -U accelerate

import torch
import soundfile as sf
from IPython.display import Audio, display
import os

# Verificar si CUDA (GPU) está disponible
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Usando dispositivo: {device}")

if device == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
else:
    print("Advertencia: Ejecutando en CPU. La generación será lenta.")

```

También se creó una función auxiliar para reproducir los audios directamente dentro del notebook. Esto permitió revisar los resultados generados sin necesidad de descargar el archivo inmediatamente.

```

# Función auxiliar para reproducir audio directamente en el notebook
def play_audio(file_path):
    if os.path.exists(file_path):
        display(Audio(file_path))
    else:
        print(f"Archivo no encontrado: {file_path}")

```

Carga del modelo de clonación de voz

Para la clonación de voz se utilizó el modelo Qwen/Qwen3-TTS-12Hz-1.7B-Base, cargado mediante la clase Qwen3TTSTModel. Este modelo permite generar voz a partir de texto utilizando como referencia un audio previo para imitar características vocales.

El modelo fue cargado en Google Colab usando la configuración automática del dispositivo. Cuando había GPU disponible, se usó un tipo de dato más optimizado para reducir el consumo de memoria y mejorar el rendimiento. Cuando no había GPU, el modelo se ejecutaba en CPU, aunque de forma más lenta.

```

from qwen_tts import Qwen3TTSTModel

# Seleccionar y cargar modelo base
model_name = "Qwen/Qwen3-TTS-12Hz-1.7B-Base"

print(f"Cargando {model_name}...")

```

```
# Cargar el modelo Base/Clone
base_model = Qwen3TTSTModel.from_pretrained(
    model_name,
    device_map="auto",
    dtype=torch.bfloat16 if device == "cuda" else torch.float32,
)

print("¡Modelo de Clonación de Voz Cargado!")
```

Uso del audio de referencia

Para que el modelo pudiera reconocer y copiar características de una voz, se utilizó un archivo de audio de referencia llamado Pablo.mp3. Este audio contenía a una persona cantando la canción “Daisy Bell”.

El archivo fue descargado previamente y luego cargado en Google Colab para poder ser utilizado por el modelo. Además del archivo de audio, se escribió manualmente el texto correspondiente al fragmento utilizado. Esto fue necesario porque el modelo utiliza tanto el audio como su transcripción para identificar mejor las características de la voz de referencia.

A partir de esa información, se creó una estructura llamada voice_clone_prompt, la cual contiene los rasgos vocales extraídos del audio.

```
# Archivo de referencia cargado en Colab
ref_audio_path = "Pablo.mp3"

# Texto correspondiente al fragmento de audio utilizado
ref_audio_text = "Daisy Daisy, cuéntame por favor, me enloqueces, muero
por ti de amor, casado sin elegancia, pues no soy de abundancia, pero tan
linda hoy te ves en nuestra bici para 2"

print(f"Clonando voz desde referencia...")

# Crear estructura del prompt de clonación
# Esto extrae las características de estilo del audio de referencia
voice_clone_prompt = base_model.create_voice_clone_prompt(
    ref_audio=ref_audio_path,
    ref_text=ref_audio_text
)
```

Generación de la voz clonada

Después de crear el prompt de clonación, se definió el texto objetivo que se quería generar con la voz clonada. En este caso, el texto utilizado fue: “La vaca Lola, La Vaca Lola, tiene cabeza y tiene cola”.

Luego se ejecutó el proceso de generación de voz utilizando el método `generate_voice_clone`. Este método recibió el texto objetivo, el idioma español y el prompt de clonación creado a partir del audio de referencia.

Como resultado, el modelo generó un nuevo archivo de audio con la frase indicada, intentando conservar características de la voz tomada como referencia. El archivo generado fue guardado con el nombre `output_cloned2.wav`.

```
# Texto que se desea generar con la voz clonada
target_text = "La vaca Lola, La Vaca Lola, tiene cabeza y tiene cola"
```

```
wavs, sr = base_model.generate_voice_clone(
    text=target_text,
    language="Spanish",
    voice_clone_prompt=voice_clone_prompt
)
```

```
# Guardar el audio clonado
output_clone_path = "output_cloned2.wav"
sf.write(output_clone_path, wavs[0], sr)
```

```
print("Referencia Original:")
print(f"(Fuente de referencia: {ref_audio_path})")
```

```
print("\nResultado Clonado:")
play_audio(output_clone_path)
```

Generación de la música infantil instrumental

Además de la voz clonada, se generó una música infantil instrumental para utilizarla como fondo. Esta música fue creada con un modelo de generación de audio a partir de texto, usando la librería `transformers` y el modelo `facebook/musicgen-small`.

Primero se instalaron y actualizaron las dependencias necesarias para la generación de audio. Luego se cargó un pipeline de tipo `text-to-audio`, que permite generar música a partir de una descripción escrita.

```
!pip install --upgrade pip
!pip install --upgrade transformers scipy accelerate
```

```
import scipy.io.wavfile
from transformers import pipeline
from google.colab import files
```

```
synthesiser = pipeline(
    "text-to-audio",
    "facebook/musicgen-small",
    device=0
)
```

)

Después se escribió un prompt detallado en inglés para indicar que la canción debía ser infantil, alegre, bailable y únicamente instrumental. Se especificó varias veces que no debía contener voz, canto ni letra, ya que la música sería usada como fondo para acompañar la voz clonada.

```
prompt = ""
```

```
Instrumental children's song only, no vocals, no singing, no lyrics, no human voice.
```

```
Happy and cheerful music for kids, playful and innocent mood.
```

```
Latin pop inspired rhythm, soft and danceable beat, suitable for children.
```

```
Use bright and friendly instruments such as ukulele, xylophone, marimba, soft piano, hand claps, light percussion, bells and gentle bass.
```

```
Simple catchy melody, repetitive and easy to remember.
```

```
Warm, joyful and energetic atmosphere, like background music for a children's video.
```

```
Clean production, soft transitions, no aggressive sounds, no distorted instruments.
```

```
Instrumental background music, around 30 seconds, happy ending.
```

```
""
```

Luego se ejecutó la generación del audio usando el prompt definido. El parámetro `do_sample` permitió obtener un resultado más creativo y `max_new_tokens` controló la duración aproximada del audio generado.

```
music = synthesiser(  
    prompt,  
    forward_params={  
        "do_sample": True,  
        "max_new_tokens": 1500  
    }  
)
```

Finalmente, el audio generado se guardó en formato `.wav` dentro del entorno de Colab y se descargó al equipo. El archivo fue generado con el nombre `musica_fondo_30s.wav`, aunque posteriormente se utilizó como música infantil de fondo bajo el nombre `musicainfantil.wav`.

```
ruta_salida = "/content/musica_fondo_30s.wav"
```

```
scipy.io.wavfile.write(  
    ruta_salida,  
    rate=music["sampling_rate"],  
    data=music["audio"]  
)
```

```
print("Audio generado:", ruta_salida)
```

```
files.download(ruta_salida)
```


Mezcla de voz y música

Una vez generados los dos audios principales, se procedió a unirlos en un solo archivo. Para esto se utilizó la librería pydub, que permite cargar, modificar, repetir y mezclar archivos de audio.

Primero se cargó el archivo de voz output_cloned2.wav y luego el archivo de música musicainfantil.wav. La voz fue tomada como audio principal, mientras que la música fue utilizada como fondo.

Antes de mezclar ambos archivos, se redujo el volumen de la música en 15 decibelios. Esto se hizo para evitar que la música tapara la voz, permitiendo que la frase generada se escuchara de forma clara.

También se igualaron la frecuencia de muestreo y la cantidad de canales de la música con los de la voz. Esto fue necesario para evitar errores o diferencias técnicas durante la mezcla.

```
!pip install pydub
```

```
from pydub import AudioSegment
from google.colab import files
```

```
# Cargar archivos
```

```
voz = AudioSegment.from_file("output_cloned2.wav")
musica = AudioSegment.from_file("musicainfantil.wav")
```

```
# Bajar volumen de la música para que quede de fondo
musica = musica - 15
```

```
# Igualar canales y frecuencia para evitar problemas al mezclar
musica = musica.set_frame_rate(voz.frame_rate).set_channels(voz.channels)
```

Repetición de la música en bucle

Como la música infantil tenía una duración aproximada de 30 segundos, se programó un bucle para repetirla automáticamente hasta cubrir toda la duración de la voz.

El código comparó la duración de la voz con la duración de la música. Si la música era más corta, se añadía nuevamente al audio de fondo hasta cubrir toda la duración necesaria. Luego, la pista musical repetida fue recortada exactamente al mismo largo que la voz.

```
# Repetir la música en bucle hasta cubrir toda la duración de la voz
duracion_voz = len(voz)
musica_bucle = AudioSegment.empty()
```

```
while len(musica_bucle) < duracion_voz:
    musica_bucle += musica
```

```
# Recortar la música al mismo largo que la voz
musica_bucle = musica_bucle[:duracion_voz]
```

Exportación del audio final

Después de ajustar la música y repetirla en bucle, se realizó la mezcla final utilizando la voz como pista principal y la música como fondo.

El resultado fue exportado en formato .wav con el nombre audio_final.wav. Finalmente, se utilizó la función de descarga de Google Colab para guardar el archivo final en la computadora.

```
# Mezclar voz + música
audio_final = voz.overlay(musica_bucle)

# Guardar resultado
ruta_salida = "audio_final.wav"
audio_final.export(ruta_salida, format="wav")

print("Audio final generado:", ruta_salida)

# Descargar
files.download(ruta_salida)
```

El archivo final contiene la voz clonada diciendo la frase seleccionada, acompañada por una música infantil instrumental de fondo. Este audio fue preparado para acompañar la animación del personaje dentro del proyecto multimedia.

Integración del audio final en Unity

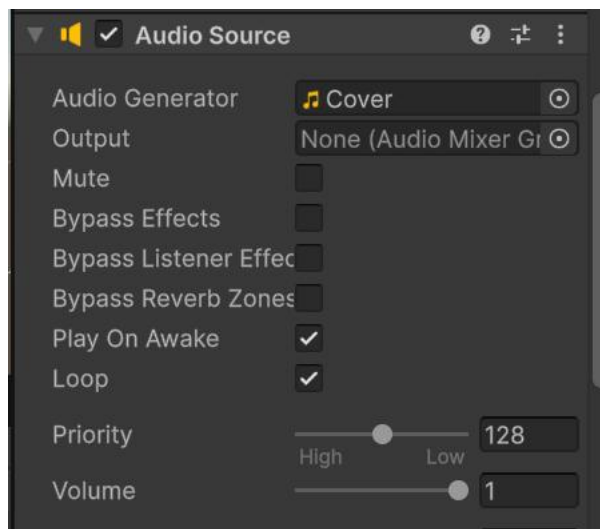
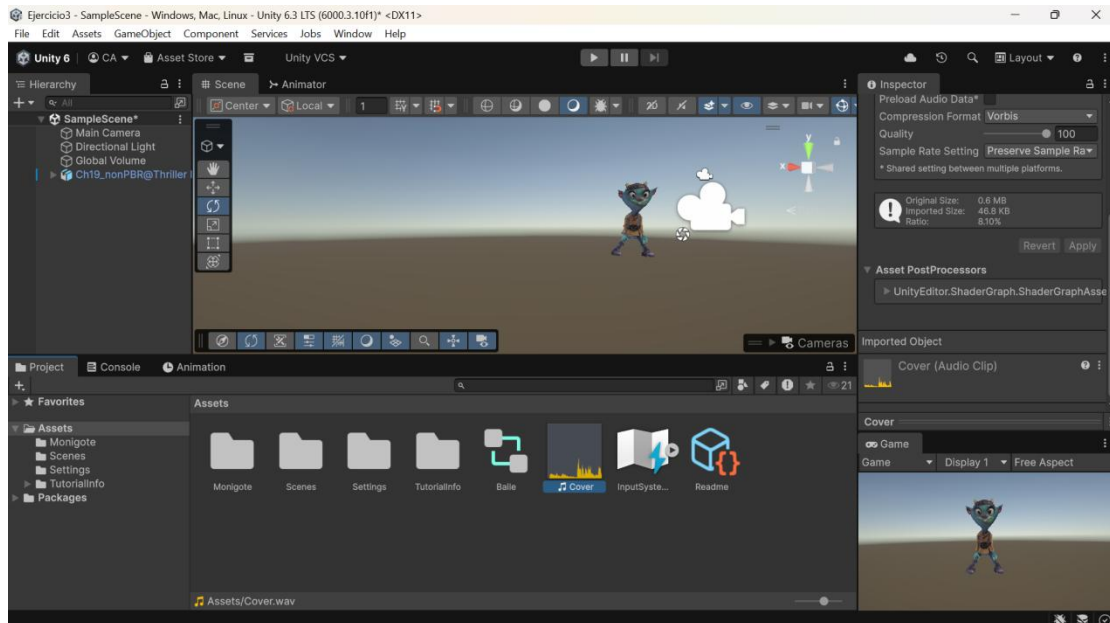
Como último paso, se importó el archivo de audio generado audio_final.wav dentro del proyecto de Unity. Para hacerlo, el archivo fue agregado a la carpeta de recursos del proyecto, permitiendo que Unity lo reconociera como un AudioClip.

Después de importar el audio, se añadió un componente Audio Source en la escena. Este componente fue utilizado para reproducir el archivo de sonido durante la ejecución del programa.

El archivo audio_final.wav se asignó al campo de audio del componente Audio Source. Luego se activó la opción de reproducción en bucle, conocida como Loop, para que el audio se repitiera continuamente mientras la escena estuviera activa.

También se configuró la reproducción automática mediante la opción Play On Awake, permitiendo que la música comenzara a sonar al iniciar la escena sin necesidad de presionar ningún botón adicional.

De esta manera, el personaje animado en Unity puede bailar de forma continua mientras el audio generado se reproduce en segundo plano. La configuración en bucle permite que la música acompañe toda la escena, manteniendo una presentación constante y sincronizada con la animación del personaje.



Resultado final

El resultado final fue una escena en Unity donde el personaje 3D importado desde Mixamo ejecuta una animación de baile en bucle. Al mismo tiempo, se reproduce el audio final generado en Google Colab, compuesto por una voz clonada y música infantil instrumental de fondo.

La integración del audio dentro de Unity permitió completar el proyecto multimedia, uniendo animación 3D, inteligencia artificial para generación de

voz y música, edición de audio y reproducción en tiempo real dentro del motor Unity.

Instalación y ejecución del proyecto (Unity WebGL)

1. Requisitos previos

- Un navegador web actualizado (Google Chrome, Microsoft Edge o Firefox).
- Un servidor local o forma de ejecución de archivos estáticos (recomendado para evitar errores de carga).
 - Ejemplo: Visual Studio Code con extensión “Live Server”, o Python instalado.

2. Estructura del proyecto

El proyecto de Unity exportado a WebGL contiene una carpeta de compilación que normalmente incluye:

- index.html
- Carpeta Build/
- Carpeta TemplateData/

Estos archivos son los necesarios para ejecutar el juego en el navegador.

3. Ejecución en local (método recomendado)

Opción A: Live Server (Visual Studio Code)

1. Instalar Visual Studio Code.
2. Instalar la extensión **Live Server**.
3. Abrir la carpeta del proyecto WebGL en VS Code.
4. Hacer clic derecho sobre el archivo **index.html**.
5. Seleccionar “**Open with Live Server**”.
6. El proyecto se abrirá automáticamente en el navegador.

Opción B: Python (servidor simple)

1. Abrir una terminal o consola en la carpeta del proyecto.
2. Ejecutar el siguiente comando:
 - `python -m http.server 8000`
3. Abrir el navegador y entrar a:
 - `http://localhost:8000`
4. Hacer clic en index.html si no se abre automáticamente.

4. Ejecución directa (no recomendada)

- También es posible abrir el archivo **index.html** directamente con doble clic.
- Sin embargo, algunos navegadores pueden bloquear la carga de recursos por seguridad, por lo que este método puede fallar.

5. Uso del proyecto

- Una vez abierto en el navegador, el proyecto se carga automáticamente.
- Se interactúa mediante teclado, mouse o controles definidos dentro del juego de Unity.
- El rendimiento puede variar según el navegador y el dispositivo utilizado.